Decision Trees Example - Problem 9.2 & Problem 9.3 (a)

> (c) 2025 Galit Shmueli, Peter C. Bruce, Peter Gedeck
>
> Code included in
>
> *Machine Learning for Business Analytics: Concepts, Techniques, and Applications in Python* (Second Edition) Galit Shmueli, Peter C. Bruce, Peter Gedeck, and Nitin R. Patel. 2025.

Add comments to each **code** cell (no comments need to be added for Markdown).

Each code cell starts with two comments:

1. Your initials and last two digits of your ID number.
2. Provide a brief explanation of the code cell. If comments are already included in the code cell, you may revise them for clarity, summarize the existing comments, or leave them unchanged if they are already appropriate. For example,

JT88

This code cell imports necessary Python packages.

In [1]:
```
import mlba
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import plot_tree, DecisionTreeClassifier, DecisionTreeRegr
import matplotlib.pylab as plt
```

```
no display found. Using non-interactive Agg backend
```

Problem 9.2 Predicting Delayed Flights.

The dataset **FlightDelays.csv** contains information on all commercial flights departing the Washington, DC area and arriving at New York during January 2004. For each flight, there is information on the departure and arrival airports, the distance of the route, the scheduled time and date of the flight, and so on. The variable that we are trying to predict is whether or not a flight is delayed. A delay is defined as an arrival that is at least 15 minutes later than scheduled.

Data Preprocessing. Transform variable day of week (DAY_WEEK) info a categorical variable. Bin the scheduled departure time into eight bins. Use these and all other

columns as predictors (excluding DAY_OF_MONTH). Partition the data into training (60%) and holdout (40%) sets.

Data Preprocessing

In [2]:
```
#VB67
#loading dataset
delays_df = mlba.load_data('FlightDelays.csv')
#displaying first five rows
delays_df.head()
```

Out[2]:

	CRS_DEP_TIME	CARRIER	DEP_TIME	DEST	DISTANCE	FL_DATE	FL_NUM	ORIG
0	1455	OH	1455	JFK	184	01/01/2004	5935	B
1	1640	DH	1640	JFK	213	01/01/2004	6155	D
2	1245	DH	1245	LGA	229	01/01/2004	7208	I
3	1715	DH	1709	LGA	229	01/01/2004	7215	I
4	1039	DH	1035	LGA	229	01/01/2004	7792	I

In [3]:
```
#VB67
# Convert the 'DAY_WEEK' column to a categorical data type for better memory
# and to indicate that it represents discrete categories (days of the week)
delays_df['DAY_WEEK'] = delays_df['DAY_WEEK'].astype('category')
```

In [4]:
```
#VB67
# Bin 'CRS_DEP_TIME' into 8 categories and convert to categorical type
delays_df['binned_CRS_DEP_TIME'] = pd.cut(delays_df.CRS_DEP_TIME, 8, labels=
delays_df['binned_CRS_DEP_TIME'] = delays_df['binned_CRS_DEP_TIME'].astype('
```

In [5]:
```
#VB67
# Define target variable as 'Flight Status' and select all other columns (ex
outcome = 'Flight Status'
predictors = [col for col in delays_df.columns
              if col not in (outcome, 'DAY_OF_MONTH')]
```

9.2.a. Fit a classification tree to the flight delay variable using all the relevant predictors. Do not include DEP_TIME (actual departure time) in the model because it is unknown at the time of prediction (unless we are generating our predictions of delays after the plane takes off, which is unlikely). Use a tree with maximum depth 8 and minimum impurity decrease = 0.01. Express the resulting tree as a set of rules.

In [6]:
```
#VB67
# Remove unnecessary columns from the list of predictors
removeColumns = ['CRS_DEP_TIME', 'DEP_TIME', 'FL_DATE', 'FL_NUM', 'TAIL_NUM'
predictors = [col for col in predictors if col not in removeColumns]
```

In [7]:
```
#VB67
# Convert categorical variables into dummy/indicator variables, define predi

X = pd.get_dummies(delays_df[predictors], prefix_sep='_')
y = delays_df[outcome]

train_X, holdout_X, train_y, holdout_y = train_test_split(X, y, test_size=0.
#Display first five rows
train_X.head()
```

Out[7]:

	DISTANCE	Weather	CARRIER_CO	CARRIER_DH	CARRIER_DL	CARRIER_MQ	C
1215	229	0	False	True	False	False	
1476	214	0	False	False	False	False	
1897	214	0	False	False	False	False	
83	214	0	False	False	False	False	
1172	213	0	False	False	True	False	

5 rows × 31 columns

In [8]:
```
#VB67
# Build and visualize a pruned decision tree model with limited depth and sp
smallClassTree = DecisionTreeClassifier(max_depth=8, min_samples_split=50, m
smallClassTree.fit(train_X, train_y)

print(f'Tree has {smallClassTree.get_n_leaves()} terminal nodes')
plot_tree(smallClassTree, feature_names=train_X.columns, filled=True, impuri
          class_names=smallClassTree.classes_)
#display
plt.show()
```

```
Tree has 2 terminal nodes
```

```
/var/folders/2h/d72vcx3n45q8sr7vv6zmq2q40000gp/T/ipykernel_4284/2113542124.p
y:10: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be sh
own
  plt.show()
```

The limited tree has only one splitting variable: Weather

If (Weather <= 0.5) then classify as Ontime.

9.2.b. If you needed to fly between DCA and EWR on a Monday at 7:00 AM, would you be able to use this tree? What other information would you need? Is it available in practice? What information is redundant?

Answer:

We cannot use this tree, because we must know the Weather. The redundant information is the day of week (Monday) and arrival airport (EWR). The tree requires knowing whether the weather was inclement or not. We may not know the weather in advance.

9.2.c. Fit the same tree as in (a), this time excluding the Weather predictor. Display both the resulting (small) tree and the full-grown tree. You will find that the small tree contains a single terminal node.

In [9]:
```
#VB67
# Exclude the 'Weather' column from the list of predictors
predictors1 = [col for col in predictors if col != 'Weather']
```

In [10]:
```
#VB67
# Build and visualize a full-grown decision tree model without limiting dept
X1 = pd.get_dummies(delays_df[predictors1], prefix_sep='_')
y1 = delays_df['Flight Status']

train_X1, holdout_X1, train_y1, holdout_y1 = train_test_split(X1, y1, test_s

# full-grown tree
fullClassTree = DecisionTreeClassifier()
fullClassTree.fit(train_X1, train_y1)

print(f'Tree has {fullClassTree.get_n_leaves()} terminal nodes')
plot_tree(fullClassTree, feature_names=train_X1.columns, filled=True, impuri
          class_names=fullClassTree.classes_)
plt.show()
```

```
Tree has 281 terminal nodes
```

```
/var/folders/2h/d72vcx3n45q8sr7vv6zmq2q40000gp/T/ipykernel_4284/3621199321.p
y:15: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be sh
own
  plt.show()
```

In [11]:
```
#VB67
# # Build a smaller decision tree with limited depth and split conditions
smallClassTree = DecisionTreeClassifier(max_depth=8, min_samples_split=50, m
# Train the model on the training dataset
smallClassTree.fit(train_X1, train_y1)

# Display the number of terminal (leaf) nodes in the tree
print(f'Tree has {smallClassTree.get_n_leaves()} terminal nodes')
# Visualize the decision tree structure
plot_tree(smallClassTree, feature_names=train_X1.columns, filled=True, impur
          class_names=fullClassTree.classes_)
# Show the plotted tree
plt.show()
```

```
Tree has 1 terminal nodes
```

```
/var/folders/2h/d72vcx3n45q8sr7vv6zmq2q40000gp/T/ipykernel_4284/2919750585.p
y:8: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be sho
wn
  plt.show()
```

9.2.c.i. How is the small tree used for classification? (What is the rule for classifying?)

Answer:

In the small tree we get a single terminal node labeled "ontime." Therefore any new flight will be classified as being "on time".

9.2.c.ii To what is this rule equivalent?

Answer:

This is equivalent to the naïve rule, which is the majority rule. In this dataset most of the flights arrived on time, and therefore the naïve rule is to classify a new flight as arriving on time.

9.2.c.iii. Examine the full-grown tree. What are the top three predictors according to this tree?

Answer:

We can visualize only the top of the tree

In [12]:
```
#VB67
# Set figure size and resolution for better visualization
plt.figure(figsize=[8, 4], dpi=300)
# Plot the top 2 levels of the full decision tree
plot_tree(fullClassTree, max_depth=2, fontsize=7, feature_names=train_X1.col
          class_names=fullClassTree.classes_)
# Display the plotted tree
plt.show()
```

```
/var/folders/2h/d72vcx3n45q8sr7vv6zmq2q40000gp/T/ipykernel_4284/2276655496.p
y:4: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be sho
wn
  plt.show()
```

The features involved in the top three decisions are

- CARRIER_US
- CARRIER_DL
- DAY_WEEK_3

9.2.c.iv. Why, technically, does the small tree result in a single node?

Answer:

The small tree results in a single node because adding splits would violate one of the constraints we used to limit tree growth.

9.2.c.v. What is the disadvantage of using the top levels of the full-grown tree as opposed to the small tree?

Answer:

Simply using the top layers of the full decision tree would be an ad hoc visual approach, and does not assure an optimal solution. Using "gridsearchCV" in Python allows us to set the parameters for limiting tree growth by assessing error on the holdout data. We did not use "gridsearchCV" in this case, opting to keep the problem simple by specifying the limiting parameters.

In [13]:
```
#VB67
# predictive power of tree
# predicted values for holdout set
pred_v = smallClassTree.predict(holdout_X1)
# confusion matrix for holdout set
mlba.classificationSummary(y_true=holdout_y1, y_pred=pred_v)
```

```
Confusion Matrix (Accuracy 0.8104)

       Prediction
 Actual delayed  ontime
delayed       0     167
 ontime       0     714
```

The logistic regression improves only marginally on the naive rule, and the simple tree we ended up with improves on it not at all, so it is likely that the predictor variables offer little predictive power. With poor predictive power and a relatively small dataset, the model-based logistic regression may do a bit better by virtue of imposing structure, as opposed to the tree, which is more at the mercy of the data and can suffer from instability.

Add-on Questions using Random Forest

Train a RF classifier, report confusion matrix and accuracy score for the holdout dataset (use X1, y1), and plot feature importance.

In [14]:
```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix, accuracy_score
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

In [15]:
```
#VB67
# Create a Random Forest classifier with 500 trees
rf = RandomForestClassifier(
    n_estimators=500,
    random_state=1,
    n_jobs=-1
)
# Train the Random Forest model on the training data
rf.fit(train_X1, train_y1)
```

Out[15]:

```
RandomForestClassifier
RandomForestClassifier(n_estimators=500, n_jobs=-1, random_state=1)
```

In [16]:

```
#VB67
pred_rf = rf.predict(holdout_X1)
```

In [17]:

```
#VB67
# Generate the confusion matrix for the holdout dataset
cm = confusion_matrix(holdout_y1, pred_rf)
# Calculate the accuracy score for the holdout predictions
acc = accuracy_score(holdout_y1, pred_rf)
# Display the confusion matrix and accuracy of the Random Forest model
print("Random Forest — Holdout Confusion Matrix:")
print(cm)
print(f"Random Forest — Holdout Accuracy: {acc:.3f}")
```

```
Random Forest — Holdout Confusion Matrix:
[[ 26 141]
 [ 77 637]]
Random Forest — Holdout Accuracy: 0.753
```

In [18]:

```
#VB67
# Create a DataFrame showing each feature and its importance score
imp = pd.DataFrame({
    "feature": train_X1.columns,
    "importance": rf.feature_importances_
}).sort_values("importance", ascending=False)

# Select the top 15 most important features
top_k = 15
imp_top = imp.head(top_k)

# Plot a horizontal bar chart of top feature importances
plt.figure(figsize=(8, 5), dpi=150)
plt.barh(imp_top["feature"][::-1], imp_top["importance"][::-1])
plt.xlabel("Importance")
plt.title("Random Forest — Top Feature Importances")
plt.tight_layout()
plt.show()
```

```
/var/folders/2h/d72vcx3n45q8sr7vv6zmq2q40000gp/T/ipykernel_4284/1532014674.p
y:14: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be sh
own
  plt.show()
```

Problem 9.3 (a) Predicting Prices of Used Cars (Regression Trees).

The dataset **ToyotaCorolla.csv** contains the data on used cars (Toyota Corolla) on sale during late summer of 2004 in the Netherlands. It has 1436 records containing details on

38 attributes, including Price, Age, Kilometers, HP, and other specifications. The goal is to predict the price of a used Toyota Corolla based on its specifications. (The example in Section 9.7 is a subset of this dataset).

Data Preprocessing. Split the data into training (60%), and holdout (40%) datasets.

9.3.a. Run a full-grown regression tree (RT) with outcome variable Price and predictors Age_08_04, KM, Fuel_Type (first convert to dummies), HP, Automatic, Doors, Quarterly_Tax, Mfr_Guarantee, Guarantee_Period, Airco, Automatic_airco, CD_Player, Powered_Windows, Sport_Model, and Tow_Bar. Set random_state=1.

9.3.a.i. Which appear to be the three or four most important car specifications for predicting the car's price?

In [19]:
```
#VB67
# Load the Toyota Corolla dataset
car_df = mlba.load_data('ToyotaCorolla.csv')

# Select relevant columns for analysis
columns = ['Price', 'Age_08_04', 'KM', 'Fuel_Type', 'HP', 'Automatic',
           'Doors', 'Quarterly_Tax', 'Mfr_Guarantee', 'Guarantee_Period',
           'Airco', 'Automatic_airco', 'CD_Player', 'Powered_Windows',
           'Sport_Model', 'Tow_Bar']

car_df = car_df[columns]

# Convert categorical variables into dummy/indicator variables
car_df = pd.get_dummies(car_df, drop_first=True)

# Display the first few rows of the cleaned dataset
car_df.head()
```

Out[19]:

	Price	Age_08_04	KM	HP	Automatic	Doors	Quarterly_Tax	Mfr_Guarantee
0	13500	23	46986	90	0	3	210	0
1	13750	23	72937	90	0	3	210	0
2	13950	24	41711	90	0	3	210	1
3	14950	26	48000	90	0	3	210	1
4	13750	30	38500	90	0	3	210	1

In [20]:
```
#VB67
# Separate features (X) and target variable (y)
X = car_df.drop(columns='Price')
y = car_df['Price']

# Split the data into training and holdout (testing) sets
train_X, holdout_X, train_y, holdout_y = train_test_split(X, y, test_size=0.
```

```
# Display the shape of the training and holdout datasets
print('Training set:', train_X.shape, 'holdout set:', holdout_X.shape)
```

```
Training set: (861, 16) holdout set: (575, 16)
```

In [21]:
```
#VB67
# Build a deep Decision Tree Regressor with default settings
deepTree = DecisionTreeRegressor(random_state=1)
# Train the regression tree on the training data
deepTree.fit(train_X, train_y)

# Print the number of terminal (leaf) nodes in the tree
print(f'Tree has {deepTree.get_n_leaves()} terminal nodes')
# Visualize the full regression tree
plot_tree(deepTree, feature_names=train_X.columns, impurity=False, filled=Tr
# Display the plotted tree
plt.show()
```

```
Tree has 743 terminal nodes
```

```
/var/folders/2h/d72vcx3n45q8sr7vv6zmq2q40000gp/T/ipykernel_4284/3218427491.p
y:6: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be sho
wn
  plt.show()
```

In [22]:
```
#VB67
# Create a DataFrame showing feature importance scores from the regression t
importance = pd.DataFrame({'features': train_X.columns,
                           'importance': deepTree.feature_importances_})
# Sort features by importance in descending order
importance.sort_values(by='importance', ascending=False)
```

Out[22]:

	features	importance
0	Age_08_04	0.844867
2	HP	0.053789
1	KM	0.049601
9	Automatic_airco	0.013358
5	Quarterly_Tax	0.006769
11	Powered_Windows	0.005221
4	Doors	0.004864
8	Airco	0.004727
12	Sport_Model	0.004459
6	Mfr_Guarantee	0.003714
7	Guarantee_Period	0.002385
13	Tow_Bar	0.002345
10	CD_Player	0.002088
3	Automatic	0.001334
15	Fuel_Type_Petrol	0.000470
14	Fuel_Type_Diesel	0.000011

From the regression tree it appears that the most important car specifications to predict the car's price are Age_08_04 (Age in months as of August 2004), HP (Horse Power), KM (accumulated kilometers on odometer) and Automatic_airco (Automatic Airconditioning(Yes=1, No=0)).

9.3.a.ii. Compare the prediction errors of the training and holdout sets by examining their RMSE error and by plotting the two boxplots. How does the predictive performance of the holdout set compare to the training set? Why does this occur?

In [23]:

```
#VB67
# Evaluate model performance on the training set
print('training set:')
mlba.regressionSummary(y_true=train_y, y_pred=deepTree.predict(train_X))
# Evaluate model performance on the holdout (testing) set
print('\nholdout set:')
mlba.regressionSummary(y_true=holdout_y, y_pred=deepTree.predict(holdout_X))
```

```
training set:

Regression statistics

Mean Error (ME) : 0.0000
Root Mean Squared Error (RMSE) : 0.0000
Mean Absolute Error (MAE) : 0.0000
Mean Percentage Error (MPE) : 0.0000
Mean Absolute Percentage Error (MAPE) : 0.0000

holdout set:

Regression statistics

Mean Error (ME) : 76.6557
Root Mean Squared Error (RMSE) : 1492.3365
Mean Absolute Error (MAE) : 1152.4852
Mean Percentage Error (MPE) : -0.3363
Mean Absolute Percentage Error (MAPE) : 11.3783
```

The RMSE of the training set is 0 since the tree is as complex as it can be and fully predicts every record. The RMSE of the holdout set, which was not involved in the overfitting, is about 1500.

9.3.a.iii. How might we achieve better holdout predictive performance at the expense of training performance?

By limiting tree growth with GridSearchCV, which assesses the performance of many different tree sizes.

9.3.a.iv. Create a smaller tree by using GridSearchCV() or RandomizedSearchCV() with cv=5 to find a fine-tuned tree. Compared to the full-grown tree, what is the predictive performance on the holdout set?

In [24]:
```
#VB67
# Before doing full grid search, try a tree of depth = 5
# smaller tree
shallowTree = DecisionTreeRegressor(max_depth=5)
shallowTree.fit(train_X, train_y)
print(f'Tree has {shallowTree.get_n_leaves()} terminal nodes')
plt.figure(figsize=[20, 4], dpi=300)
plot_tree(shallowTree, feature_names=train_X.columns, impurity=False, fillec
          fontsize=7)
plt.show()
```

```
Tree has 30 terminal nodes
/var/folders/2h/d72vcx3n45q8sr7vv6zmq2q40000gp/T/ipykernel_4284/1209277528.p
y:9: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be sho
wn
  plt.show()
```

In [25]:
```
#VB67
# Evaluate shallow regression tree performance on the training set
```

```
print('training set:')
mlba.regressionSummary(y_true=train_y, y_pred=shallowTree.predict(train_X))
# Evaluate shallow regression tree performance on the holdout (testing) set
print('\nholdout set:')
mlba.regressionSummary(y_true=holdout_y, y_pred=shallowTree.predict(holdout_
```

```
training set:

Regression statistics

Mean Error (ME) : -0.0000
Root Mean Squared Error (RMSE) : 1028.0279
Mean Absolute Error (MAE) : 773.2770
Mean Percentage Error (MPE) : -1.0039
Mean Absolute Percentage Error (MAPE) : 7.6715

holdout set:

Regression statistics

Mean Error (ME) : 52.8857
Root Mean Squared Error (RMSE) : 1160.9679
Mean Absolute Error (MAE) : 914.1468
Mean Percentage Error (MPE) : -0.6477
Mean Absolute Percentage Error (MAPE) : 9.2441
```

For the shallower tree, the RMSE values of training and holdout sets are comparable. Compared to the deep tree, the RMSE of the holdout set is lower. This shallower tree does much worse on the training set than the deep tree but does better in predicting rows in the holdout set. This indicates a more robust model.

In [26]:
```
#VB67
# Define a grid of hyperparameters for the regression tree
param_grid = {
    'max_depth': [5, 10, 15, 20, 25],
    'min_impurity_decrease': [0, 0.001, 0.005, 0.01],
    'min_samples_split': [10, 20, 30, 40, 50],
}
# Set up GridSearchCV to find the best hyperparameters using 5-fold cross-va
gridSearch = GridSearchCV(DecisionTreeRegressor(random_state=1),
                          param_grid, cv=5, n_jobs=-1)
# Fit the grid search on the training data
gridSearch.fit(train_X, train_y)
# Display the best combination of hyperparameters
print('Initial parameters: ', gridSearch.best_params_)
```

```
Initial parameters:  {'max_depth': 10, 'min_impurity_decrease': 0, 'min_samp
les_split': 20}
```

In [27]:
```
#VB67
# Define a refined grid of hyperparameters for the regression tree
param_grid = {
    'max_depth': [5, 6, 7, 8, 9, 10, 11, 12, 13],
    'min_impurity_decrease': [0, 0.001, 0.002, 0.003, 0.004, 0.005,
                              0.006, 0.007],
    'min_samples_split': [17, 18, 19, 20, 21, 22, 23],
```

```
}
# Set up GridSearchCV to find the best hyperparameters using 5-fold cross-va
gridSearch = GridSearchCV(DecisionTreeRegressor(random_state=1),
                          param_grid, cv=5, n_jobs=-1)
# Fit the grid search on the training data
gridSearch.fit(train_X, train_y)
# Display the best combination of hyperparameters
print('Improved parameters: ', gridSearch.best_params_)

# Get the optimized regression tree based on the best hyperparameters
optRegTree = gridSearch.best_estimator_

# Create a DataFrame showing feature importance from the deep tree
importanceRT = pd.DataFrame({'features': train_X.columns,
                             'importance': deepTree.feature_importances_})

# Print the number of terminal (leaf) nodes in the optimized tree
print(f'Tree has {optRegTree.get_n_leaves()} terminal nodes')
```

```
Improved parameters:  {'max_depth': 6, 'min_impurity_decrease': 0, 'min_samp
les_split': 21}
Tree has 31 terminal nodes
```

In [28]:

```
#VB67
# Set figure size and resolution for better visualization
plt.figure(figsize=[16, 7], dpi=300)
# Plot the optimized regression tree with all features
plot_tree(optRegTree, feature_names=train_X.columns, filled=True, fontsize=4
# Display the plotted tree
plt.show()
```

```
/var/folders/2h/d72vcx3n45q8sr7vv6zmq2q40000gp/T/ipykernel_4284/1299930031.p
y:3: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be sho
wn
  plt.show()
```

In [29]:

```
#VB67
# Compute regression metrics for the optimized tree on training and holdout
pd.DataFrame({
    'training': mlba.regressionMetrics(y_true=train_y, y_pred=optRegTree.pre
    'holdout': mlba.regressionMetrics(y_true=holdout_y, y_pred=optRegTree.pr
}).transpose().round(3)
```

Out[29]:

	ME	RMSE	MAE	MPE	MAPE
training	0.000	1100.938	799.410	-1.008	7.685
holdout	23.704	1240.202	950.009	-1.062	9.517

In [30]:

```
#VB67
# Import necessary modules for randomized hyperparameter search
import scipy
from sklearn.model_selection import RandomizedSearchCV

# Define a distribution of hyperparameters for the regression tree
param_dist = {
```

```
    'max_depth': list(range(1, 25)),
    'min_impurity_decrease': scipy.stats.expon(scale=1),
    'min_samples_split': list(range(2, 50)),
}
# Set up RandomizedSearchCV with 200 random combinations and 5-fold CV
randSearch = RandomizedSearchCV(DecisionTreeRegressor(random_state=432), par
                                n_jobs=-1, n_iter=200, random_state=123)
# Fit the randomized search on the training data
randSearch.fit(train_X, train_y)
# Display the best combination of hyperparameters found
print('Best parameters: ', randSearch.best_params_)

# Get the optimized regression tree based on RandomizedSearchCV
optRegTree = randSearch.best_estimator_

# Compute and display regression metrics for training and holdout sets
pd.DataFrame({
    'training': mlba.regressionMetrics(y_true=train_y, y_pred=optRegTree.pre
    'holdout': mlba.regressionMetrics(y_true=holdout_y, y_pred=optRegTree.pr
}).transpose().round(3)
```

```
Best parameters:  {'max_depth': 6, 'min_impurity_decrease': np.float64(0.245
94585858074333), 'min_samples_split': 8}
```

Out[30]:

	ME	RMSE	MAE	MPE	MAPE
training	-0.0	932.784	713.840	-0.880	7.161
holdout	31.5	1177.700	919.125	-0.974	9.339

In [ ]: